



Pineapple: Unifying Multi-Paxos and Atomic Shared Registers

Tigran Bantikyan, *Northwestern*; Jonathan Zarnstorff, *unaffiliated*;
Te-Yen Chou, *CMU*; Lewis Tseng, *UMass Lowell*; Roberto Palmieri, *Lehigh University*

<https://www.usenix.org/conference/nsdi25/presentation/bantikyan>

This paper is included in the
Proceedings of the 22nd USENIX Symposium on
Networked Systems Design and Implementation.

April 28–30, 2025 • Philadelphia, PA, USA

978-1-939133-46-5

Open access to the Proceedings of the
22nd USENIX Symposium on Networked
Systems Design and Implementation
is sponsored by



Pineapple: Unifying Multi-Paxos and Atomic Shared Registers*

Tigran Bantikyan
Northwestern

Jonathan Zarnstorff
Unaffiliated

Te-Yen Chou
CMU

Lewis Tseng
UMass Lowell

Roberto Palmieri
Lehigh University

Abstract

Linearizable storage systems reduce the complexity of developing correct large-scale customer-facing applications, in the presence of concurrent operations and failures. A common approach for providing linearizability is to use consensus to order operations invoked by applications. This paper explores designs that offload operations (from the consensus component) to improve overall performance.

This paper presents Pineapple, which uses logical timestamps to unify Multi-Paxos and atomic shared registers so that any node in the system can serve read and write operations. Compared to Multi-Paxos (or leader-based consensus), Pineapple reduces bottlenecks at the leader. Compared to Gryff, which unifies EPaxos and atomic shared registers, Pineapple has better performance because Pineapple has “*non-blocking* operation execution.”

Our evaluation shows that Pineapple improves both throughput and tail latency, compared to state-of-the-art systems (e.g., Gryff, Multi-Paxos, EPaxos), in both wide-area networks and local-area networks. We also integrate Pineapple with etcd. In a balanced workload, Pineapple reduces median latency by more than 50%, compared to the original system that uses an optimized version of Raft.

1 Introduction

Large-scale customer-facing applications increasingly rely on linearizable storage systems. These systems ensure that a service is highly available and consistent in the presence of concurrent operations and node crashes. Modern applications prefer strong consistency over weaker consistency, because it simplifies application logic, which then significantly lowers the effort in development and maintenance [11, 15, 17, 19].

One common solution for developing linearizable storages is using consensus. Multi-Paxos [36, 39] and Raft [53],

two popular leader-based consensus algorithms, have been adopted in many production storage systems, e.g., Google Spanner [17], Microsoft Azure Storage [11], Redis [2], RethinkDB [4], CockroachDB [3], and etcd [1]. To improve performance, this paper explores approaches that offload reads and writes from the leader to followers (or non-leaders).

Targeted Workloads. We focus on linearizable storages designed for large-scale web applications. For scalability and availability, these applications often rely on variations of key-value storages, NoSQLs, or NewSQLs, whose performance could be improved using our design.¹ Most modern storages support reads and writes on a single key-value pair, e.g., Cassandra [35], etcd [1], Redis [2], and Memcached [7, 52].

Example large-scale web applications that heavily use such single-key reads or writes include Facebook inbox search [35], WeChat’s user account management [73], and Microsoft’s OneDrive [15]. In these applications, the program logic and data models allow the developers to use single-key operations efficiently. For example, WeChat creates table(s) as per user account [73], and Microsoft uses a versioned object storage as the backend for OneDrive [15].

In fact, the need to optimize read and write performance can be found in many real-world systems. For example, WeChat designs PaxosStore [73] that uses a combination of quorum and leader lease to reduce latency for reads and writes. Google’s Spanner [17] and Microsoft’s Giza [15] use variations of techniques to achieve similar goals (e.g., Spanner’s TrueTime and Giza’s adoption of Fast Paxos [38]).

Performance studies from Facebook [7, 9] also support the goal of improving read and write performance: (i) Only 0.2% of operations involve a write in their distributed data store for social graph TAO; and (ii) 97% of operations are reads, writes, or deletes in Facebook’s usage of RocksDB, a key-value store.

Pineapple. Motivated by these observations, we propose Pineapple, a high-performance *linearizable key-value storage*

*The work was partially done when the first three authors were with Clark University. First two authors have equal contribution. This material is based upon work supported by the National Science Foundation under Grant No. CNS-2045976 and CNS-2449640.

¹In typical NoSQL databases, an entire row can be treated as a single key-value pair. Hence, our design could be compatible with data models in similar systems. For conciseness, the paper is focused on key-value storages.

that offloads reads and writes from the leader of Multi-Paxos to followers. As in [37,43,46,61], we assume that an operation touches a single key or a set of keys, where each key uniquely identifies an object (or a piece of data). With a slight abuse of terminology, we use “key” and “object” interchangeably. Pineapple supports three operations (definitions in §2.1):

- (i) reading the value from a single key,
- (ii) writing a value to a single key, and
- (iii) performing a one-shot transaction [34], which either accesses or updates the values of a set of keys, or provides a stronger synchronization (e.g., read-modify-write, fetch-and-add, compare-and-swap), or both.

Pineapple: Blind Writes and One-shot Transactions. The single-key write in Pineapple is in fact a “blind write,” which overwrites an existing key-value pair. While not as powerful as conditional writes, blind writes are still applicable to many web applications. For instance, many storage systems, such as Cassandra, Spanner, HBase, RocksDB, Redis, and etcd, support blind writes. Moreover, useful features for web applications can be implemented using blind writes. Examples include versioned objects for Microsoft Giza [15] and Amazon S3, and data structures with associative and commutative properties (e.g., data structures with statistical or probabilistic properties like Histogram and Hyperloglog) [12].

Pineapple supports one-shot transactions, which can be represented as *stored procedures* [34,48,49]. In these transactions, clients (or applications) send the transaction logic to servers, and the servers execute them *atomically*, without client interaction during the process. Additionally, the transactions’ input and output key sets are known a priori. Therefore, servers can execute the transactions (once they are ordered) without further interaction with the clients.

While less powerful than general transactions, one-shot transactions are sufficient for many web applications [10,49]. For example, one-shot transactions can automatically update multiple users’ accounts (e.g., money transfers) on payment platforms. They can also enable a scan or range query for files on cloud object-based storage. A key-value store, which provides only single-key reads/writes, and Gryff [10], which provides only single-key reads/writes/read-modify-writes, are not sufficient for supporting these operations. An application either has to sacrifice guarantees (e.g., relaxed consistency when doing a range query) or use general transactional storage (potentially with lower performance on single-key operations).

Pineapple: Key Techniques. Pineapple uses a quorum-based leaderless approach (ABD [8]) to serve reads and writes on a single key-value pair and relies on a leader-based consensus (Multi-Paxos) to agree on the order of one-shot transactions. Even though it is *not* as intuitive to combine a leaderless design with a leader-based one, such unification not only simplifies the overall design, but also improves performance.

Interestingly, we do not need a completely new design to

unify the two components. In particular, we use a mature technique – logical timestamps – to unify Multi-Paxos and ABD. The main technical contribution is our insight to combine them together so that they can work seamlessly together, even in the period of leader change or leader failure.

Pineapple: Highlights. Pineapple provides the following features that would favor widespread adoption.

- Pineapple offloads single-key reads and writes to follower nodes. This helps reduce tail latency for reads and writes and increase overall throughput.
- In a 3-node system, Pineapple guarantees that single-key reads complete in 1 round-trip time (RTT). With more nodes, reads can complete in 1 RTT if there is no concurrent one-shot transaction or write on the same key.
- Pineapple supports one-shot transactions that can access or update the values of a set of keys and provide stronger synchronization power.
- All operations supported by Pineapple have “*non-blocking* execution,” i.e., a node can execute any operation, after communicating with the leader or a quorum.

These features jointly allow Pineapple to improve performance over state-of-the-art systems, e.g., Gryff [10], Multi-Paxos, PQR [13], EPaxos [46], in both wide-area networks and local-area networks over a wide spectrum of workloads.

Non-blocking operation execution is a natural feature in consensus algorithms like Multi-Paxos and Raft. To improve performance, recent systems often choose to separate ordering and execution phases, causing the executions of some operations *blocking*. For example, EPaxos and Gryff need to resolve dependency before execution. PQR needs to wait for committed log entries before executing reads.

As we will see in §4, such blocking operation executions cause longer tail latency and reduced throughput. Moreover, the blocking nature is not ideal for practical deployment, because a slow node (e.g., due to intermittent connection issues, misconfigurations or garbage collection events [23,24,50]) could impact latency and throughput unexpectedly.

Pineapple also has one appealing feature for practitioners. Pineapple is built upon mature techniques that are well understood and tested, e.g., Mutli-Paxos, ABD (or quorum-based approach), and logical timestamps. Fundamentally, Pineapple adds one more round of communication into Multi-Paxos so that it could interact with ABD atomic registers. Therefore, we believe that our design of Pineapple could potentially be more straightforward to understand and integrate with production systems that use leader-based consensus.

Existing Solutions. There are roughly four categories of solutions on improving the performance of consensus-based linearizable storages: (i) a completely new consensus algorithm that relies on different assumptions and models, e.g., [6,23,24,37,44,46,54,55], (ii) a variant of existing consensus algorithm that better utilizes new hardware such as RDMA or SDN, e.g., [5,40,45], (iii) an engineering or software solution

	Latency			(Fast) Quorum Size	Non-blocking Exe.	Limitations
	Read	Write	Other			
Multi-Paxos [36]		1/2		$\lfloor n/2 \rfloor + 1$	all ops	leader bottleneck
PQR [13]	1/1	1/2	1/2	$\lfloor n/2 \rfloor + 1$	non-read ops	leader bottleneck & read tail latency
EPaxos [46]		1/2		$f + \lfloor (f+1)/2 \rfloor$	-	long tail latency
Gryff [10]	1/2	2/2	2/3	$f + \lfloor (f+1)/2 \rfloor$	reads and writes	lower throughput & no support of multi-key ops
Pineapple (this work)	1/2	2/2	2/3	$\lfloor n/2 \rfloor + 1$	all ops	3-RTT one-shot transactions

Table 1: **Pineapple vs. closely related systems.** • All of the systems tolerate up to f crashed nodes with $n = 2f + 1$ nodes.

- The first number in Latency columns indicates the RTTs in the case of no conflict (for EPaxos, Gryff, and Pineapple’s read) or the RTT in the case when the operation is served from a leader node (for Multi-Paxos, PQR and Pineapple’s write and one-shot transactions, and the second is the RTT with conflict (for EPaxos, Gryff, and Pineapple’s read) or the RTT in the case when the request is served from a follower node (for Multi-Paxos, PQR’s writes and Pineapple’s one-shot transactions. “Other” represents RMW for Gryff and one-shot transactions for other systems.
- The second column specifies the quorum size for Multi-Paxos, PQR, and Pineapple and the fast quorum size for EPaxos and Gryff.
- An operation has “*non-blocking* execution” if it can be executed immediately upon learning a decision of its ordering.

to scale an existing consensus algorithm, e.g., [14, 61, 64, 69], and (iv) a new algorithmic component that is integrated with an existing consensus algorithm, e.g., [10, 13, 31, 37, 38].

Table 1 compares key features between Pineapple and closely related systems. (More discussion on other related work in §5.) Pineapple belongs to category (iv). Two notable recent systems in this category are PQR [13] and Gryff [10]. PQR improves Multi-Paxos’s performance by offloading reads to followers. Gryff allows any node to perform reads, writes and Read-Modify-Writes (RMWs) by unifying EPaxos [46] and atomic shared registers [8, 30]. Both demonstrate improved latency or throughput over prior solutions such as Multi-Paxos and EPaxos under workloads for web applications. Compared to Pineapple, there are a few limitations in PQR and Gryff:

- In PQR, only reads are offloaded to followers; hence, other operations can still saturate the leader.
- In PQR, reads have “*blocking* execution,” i.e., reads may not be executed immediately and have to wait for a committed entry from the leader. The effect of blocking executions, which leads to longer latency, can be observed even during a light workload without leader change.
- Gryff’s throughput is generally lower than *both* EPaxos and Multi-Paxos due to its more complicated execution logic and extra message overhead.
- Gryff’s operations can *only* access or update a *single* key. In other words, it doesn’t support one-shot transactions. Many operations used by modern web applications are not supported by Gryff. (More details in §2.3.)
- Gryff’s RMWs have “*blocking* execution,” i.e., the execution of RMW needs to wait until all the prior dependency have been resolved and executed. This is because Gryff uses a variant of EPaxos to handle RMWs. Our evaluation shows that such blocking execution affects both tail latency and throughput.

Pineapple: Performance. We evaluate Pineapple in both local-area networks (LAN) and wide-area networks (WAN). Our evaluation shows that Pineapple improves performance

in both cases. In both WAN and LAN, Pineapple has roughly 10% to 20% of improvement over the closest competitor (i.e., EPaxos and PQR) in throughput when RMW ratio is less than 20%. As for tail-latency in WAN, Pineapple generally has better read tail latency than Multi-Paxos and PQR with low-conflict workloads, in 3-node and 5-node deployments.

We also integrate Pineapple with etcd [1], a backend system used by many large-scale production systems such as Kubernetes. We replace its consensus layer (powered by Raft [53]) by Pineapple. Pineapple improves median latency by more than 50% in LAN, under a YCSB balanced workload [16]. This demonstrates that our design effectively improves latency in an optimized production system.

Outline. We first discuss the preliminaries in §2. We then present our design of Pineapple and its analysis in §3. We evaluate Pineapple in §4. Related work is presented in §5.

2 Preliminaries and Background

2.1 System Model and Shared Objects

This paper is focused on systems consisting of n nodes (or servers) in a point-to-point communication network. Up to f nodes may fail by crashing, e.g., stopping execution at any point of time. We assume $n = 2f + 1$.

The system is assumed to be *asynchronous*. That is, there is no upper bound on the time a message takes to be delivered, and there is no upper bound on the relative speed at which nodes proceed. Due to the asynchrony assumption, a failure is not detectable by others. Pineapple ensures safety with asynchrony. Reads and writes can be completed under asynchrony, due to the adoption of ABD registers [8]. One-shot transactions satisfy liveness under *partial synchrony* [22] – when there is a stable leader that can communicate with a quorum, then one-shot transactions are guaranteed to be completed.

Shared Objects. The system consists of m shared objects (i.e., key-value pairs) that can be accessed and updated concurrently.² Each object is uniquely identified by a key (or an

²The assumption of m is only for easiness of presentation. In practical

identifier). Pineapple provides the following three types of operations. Any client connects to a Pineapple node can use the interface to access or update any of the m shared objects:

- Read(k): it returns the value of key k ;
- Write(k, v): it updates the value of key k to value v ; and
- OT($f(\cdot), K_{input}, K_{output}$): it takes three parameters – a function $f(\cdot)$ and two sets of keys – and it atomically performs the following three steps:
 - (i) read the values of input key set, i.e.,
 $V_{input} \leftarrow \text{values of key set } K_{input}$
 - (ii) compute the value of output key set, i.e.,
 $V_{output} \leftarrow f(V_{input})$
 - (iii) update the values of key set K_{output} to V_{output} .

By definition, $|V_{input}| = |K_{input}|$ and $|V_{output}| = |K_{output}|$, where $|S|$ denotes the cardinality of a set S . K_{input} and K_{output} may have different sizes and may not have any overlap.

This definition of one-shot transactions captures a wide range of “transactions” used in modern large-scale applications. For example, $y \leftarrow x + 1$ can be represented as OT($f(\cdot), \{x\}, \{y\}$), where $f(v) = v + 1$.

Another common usage is “conditional write.” For example, the following function can be used for a one-shot transaction that implements an atomic counter that starts with a default value 0 and increments by 1 when invoked:

$$f(v) = \begin{cases} v + 1, & \text{if } v \text{ is non-empty} \\ 0, & \text{otherwise} \end{cases}$$

Here we assume that a returned value is empty (or null), if an object is not initialized yet.

The function $f(\cdot)$ can take multiple inputs, which is useful to support richer semantics like compare-and-swap. (An example can be found in §2.3.) An alternative view is that one-shot transactions can perform the If/Then/Else logic over any subset of the m objects “atomically.”

Linearizability. Pineapple achieves linearizability [30], which requires that concurrent operations appear to execute in some total order O such that (i) O is consistent with the interface (i.e., it is legal with respect to the semantics), and (ii) O is consistent with the real-time ordering constraint. That is, for any operation o_1 completes before the invocation of operation o_2 , o_1 must appear before o_2 in the total order O .

2.2 Ordering Read/Writes w/o Consensus

Herlihy’s wait-free hierarchy [29] defines different synchronization power of shared objects in asynchronous systems. For example, atomic shared registers that provide read and write operations on a single key have the weakest synchronization power, whereas consensus objects provide the strongest power. This hierarchy explains why there exists a non-blocking linearizable solution for ordering reads and writes for atomic registers, but not for consensus objects.

systems, objects can be dynamically created or deleted.

The difference between atomic register and other objects in the wait-free hierarchy is that each write completely defines the state of the single object *without* any reliance on a previous state. As a result, the total ordering does not have to be “stable.” In other words, the ordering does *not* need to be observed by all nodes. Only the operations that are required by real-time ordering constraints need to be observed by all nodes.

An implication of such an observation is that a node i can safely apply a write, say o_{24} , which comes with the largest logical timestamp that i has seen so far. Node i can apply o_{24} , even if it does not know any prior writes. Suppose at some later time, i receives an earlier write, say o_8 , it can safely ignore that write because it already has a more recent state (as defined by o_{24}). This is why the usage of logical timestamp is sufficient for atomic shared registers. Note that because of asynchrony, operations may be received in an arbitrary order.

Multi-Writer ABD. We briefly discuss the multi-writer variant [42] of ABD [8], which allows multiple concurrent writer clients to read or write a single key. It associates a tag $t = (ts, id)$ with each key where ts is a logical timestamp and id is the identifier of the writer proxy, i.e., the ID of the proxy node that handles the write for the client. Writes and reads can be ordered lexicographically using the associated tags, to obtain the desired total ordering for linearizability.

For each key, a node stores a value v and the tag t where t is the largest tag a node has observed so far. Both reads and writes have two phases – get and put phase. In the get phase, reads learn the value to be returned, whereas writes learn the largest tag from a quorum. In the put phase, both operations need to ensure that a quorum has obtained the information – this phase ensures real-time ordering.

A read begins with reading the tag and value from a quorum. Upon receiving responses, the reader proxy then retrieves the value v^{max} with the largest tag t^{max} from them. Then the proxy transmits (v^{max}, t^{max}) to a quorum and waits for acknowledgments. Once a quorum of acknowledgments is collected, the proxy returns v^{max} to the reader client.

A writer proxy p follows the same two-phase structure. The difference is that in the put phase, the proxy transmits the value to be written with a new tag $t = (t^{max}.ts + 1, p)$. Upon receiving a quorum of acknowledgement of the second phase, the proxy lets the writer client know that the write has been completed. Since there is no guarantee that all the writes will be observed in the same order at each node, the ordering observed by each node at any point of time is “non-stable.” That is, the order might change in the future.

Consensus and SMR. On the contrary, consensus or state-machine replication (SMR) [59] requires a “stable” ordering. More concretely, the nodes have to agree on the same sequence of operations, so the state machine at each node can apply these operations one by one starting from the beginning of the sequence. Since the operations are deterministic, nodes are guaranteed to obtain the same state upon the execution

of the j -th operation in the sequence. Here, the sequence of operations is exactly the total ordering O required by the linearizability correctness property [30].

The reason that SMR’s ordering is “stable” is that the position of an operation in the sequence will not change; otherwise, the linearizability guarantee could be violated. In comparison, the ordering of shared registers may change due to asynchrony. In the example above, node i may initially think that o_{24} is the first write. It is only until that i receives o_8 , it learns that o_{24} should occur later in the ordering. However, because of the semantic of shared registers, such a non-stable ordering still suffices for linearizability.

The difference between stable and non-stable ordering also makes it difficult to use atomic shared registers to implement an SMR system.

2.3 Challenges

RMW and Gryff’s solution. Gryff is focused on one-shot transactions that access only a single key, namely RMWs. RMWs require a stable ordering because its outcome depends on a previous state. Using the terminology introduced in §2.1, a previous state is the values of the keys specified in the second parameter K_{input} . Without a stable ordering of operations, the outcome of applying RMWs may lead to inconsistency.

More formally, suppose $K_{input} = K_{output} = \{k\}$ and $f(\cdot)$ defines a compare-and-swap (CAS):

$$f(old, new) = \begin{cases} new, & \text{if value of } k = old \\ \text{value of } k, & \text{otherwise} \end{cases}$$

Consider the case of a write o_8 that updates the value of key k to 8, and a RMW o_{24} with $f(\text{empty value}, 24)$. Now the order of how nodes apply these two operations matters. If the ordering is not stable, it is possible that node i applies o_8 before o_{24} and node j applies o_{24} before o_8 . As a result, two nodes will have divergent states, even if they eventually receive and apply both operations.

Gryff [10] uses a novel design, called Consensus-After-Register timestamps (or carstamps), to enforce the stable ordering of RMWs, while allowing writes and reads to order in a non-stable manner. Gryff uses ABD [8] to order reads and writes (the “register” part), and uses EPaxos to order RMWs (the “consensus” part). The RMWs are ordered after some writes (the “after” part).

With carstamp, Gryff then uses EPaxos to keep track of dependency and execution order. In the prior example, assume o_{24} has observed o_8 . By design, every node that applies o_{24} must wait until it has applied o_8 . This ensures linearizability, but results into *blocking* execution in Gryff’s RMWs.

Multi-Key Operations. Next, we discuss why it is non-trivial to extend ABD and Gryff to handle operations that may access or update multiple keys, namely multi-key operations. Figure 1 presents an example that shows why reading and writing each key separately using ABD is insufficient to support scan

operations, i.e., an operation that returns the value of multiple keys. In the discussion below, let $\text{scan}(x, y)$ represent a scan operation that returns the values for keys x and y atomically.

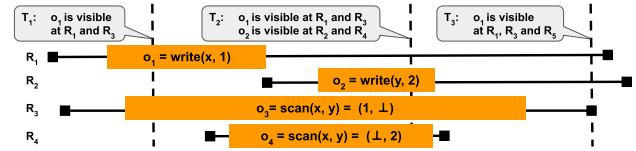


Figure 1: **An example of four operations in a five-node system.** The left dot and right dot represent the invocation time and completion time, respectively.

In this strawman example, a $\text{scan}(x, y)$ performs two ABD reads in parallel – one reading key x and the other reading key y . Suppose there are five replicas (R_5 is not shown for brevity) and four operations o_1 to o_4 such that:

- At time T_1 : the write o_1 is visible only at R_1 and R_3 ;
- At time T_2 : the write o_1 is visible only at R_1 and R_3 , and the write o_2 is visible only at R_2 and R_4 ;
- At time T_3 : the write o_1 is visible at R_1, R_3 and R_5 ; This is because the put phase of the ABD reads of o_3 helps propagate the value to a quorum, even though the operation o_1 has not completed at this point.

Now, consider the two scans:

- Scan o_3 returns $(x, y) = (1, \perp)$. This is possible if o_3 completes the get phase of two parallel ABD reads at T_1 with quorum R_1, R_3, R_5 .
- Scan o_4 returns $(x, y) = (\perp, 2)$. This is possible if o_4 completes the get phase of the two parallel ABD reads at T_2 with quorum R_2, R_4, R_5 . These nodes only know o_2 but not o_1 , as at T_2 , only R_1 and R_3 know o_1 .

Consequently, we have “incomparable” operations. That is, in the total ordering that one needs to construct for linearizability, o_3 cannot be placed before nor after o_4 . This is because such an incomparable scenario is impossible on a single machine.

This example can be extended for any algorithm, including Gryff, that does not perform coordination between different keys. Pineapple addresses this issue by relying on the leader to decide the order of multi-key operations. For example, if R_3 is the (Multi-Paxos) leader which orders o_3 before o_4 , then o_4 will return $(1, 2)$, making the two operations comparable.

3 Pineapple: Design and Analysis

This section presents the design and analysis of Pineapple. We focus on the operations on a single key k and the common case when there is no leader change. That is, the one-shot transactions defined in §2.1 have $K_{input} = K_{output} = \{k\}$. Note that in this case, Pineapple provides the same semantic as Gryff. Hence, we will use “RMWs” and “one-shot transactions” interchangeably in the following presentation. Appendix A presents the generalization to multiple keys and how Pineapple handles the case when leader changes.

Algorithm 1 Pineapple: ABD Component at Node N_i

Write(key, value) invoked by Writer Proxy N_i :

```
1: // Get phase: obtain tag
2: send (GET-TAG, key) to all
3: wait until receiving (ACK-TAG, key, psj) from quorum Q
4: psmax = (tagmax, slotmax) is largest pstamp received from all  $j \in Q$ 
5: psnew ← ((tagmax.ts + 1, i), slotmax)
6: // Put phase: propagate write request
7: send (PUT, key, psnew, value) to all
8: wait until receiving (ACK-PUT, key) from quorum
```

Read() invoked by Reader proxy N_i :

```
9: // Get phase: obtain value and tag
10: send (GET-VALUE, key) to all
11: wait until receiving (ACK-VALUE, key, valuej, psj) from quorum Q
12: psmax = (tagmax, slotmax) ← largest pstamp received from all  $j \in Q$ 
13: vmax ← value associated with psmax
```

```
14: // Put phase: propagate learned value
```

```
15: send (PUT, key, vmax, psmax) to all
16: wait until receiving (ACK-PUT, key) from quorum
17: return vmax
```

Background Handlers at node N_i :

```
18: Upon receiving (GET-TAG, key) from  $N_j$ :
19: send (ACK-TAG, key, Storagei[key].ps) to  $N_j$ 
20: Upon receiving (PUT, key, psj, valuej) from  $N_j$ :
21: if Storagei[key].ps < psj then
22:   Storagei[key].ps ← psj
23:   Storagei[key].value ← valuej
24: send (ACK-PUT, key) to  $N_j$ 
25: Upon receiving (GET-VALUE, key) from  $N_j$ :
26: send (ACK-VALUE, key, Storagei[key].value, Storagei[key].ps) to  $N_j$ 
```

Algorithm 2 Pineapple: Multi-Paxos Component at Leader N_l

RMW($f(\cdot)$, {key}, {key}) invoked by Pineapple Leader N_l :

```
1: // Get phase: obtain most recent values
2: send (GET-VALUE, key) to all
3: wait until receiving (ACK-VALUE, key, valuej, psj) from quorum Q
4: vmax ← value w/ largest psmax received from all  $j \in Q$  ▷  $l \in Q$ 
5: vnew ←  $f(v^{\max})$  ▷ Apply RMW
6: psnew ← (psmax.tag, slotl + 1) ▷ Increment slot index
7: // Put phase: propagate output
8: send (PUT, key, vnew, psnew) to all
9: wait until receiving (ACK-PUT, key) from quorum
```

3.1 Pineapple: Design

Pstamps: Timestamp for Pineapple. Similar to how Gryff uses carstamps, Pineapple uses a variation of timestamp – pstamps (which stands for Pineapple timeSTAMPS). Pstamps is a composition of tag used by ABD and the slot index uses by Multi-Paxos. Formally, pstamp p is a tuple (tag, slot), where

- tag is an ABD tag – a tuple (ts, id) such that ts is the logical timestamp and id is the ID of the writer proxy;
- slot is a positive integer that represents the slot index of the Multi-Paxos component.³

As in [8, 10, 42, 63], each write or RMW is associated with a unique pstamp, and Pineapple uses pstamps to determine the order of operations. These pstamps can be ordered lexicographically. That is, for two pstamps $p = (tag_p, slot_p)$ and $p' = (tag_{p'}, slot_{p'})$, we say that $p > p'$ if and only if:

- $tag_p.ts > tag_{p'}.ts$;
- $tag_p.ts = tag_{p'}.ts$ and $tag_p.id > tag_{p'}.id$; or
- $tag_p.ts = tag_{p'}.ts$ and $tag_p.id = tag_{p'}.id$ and $slot_p > slot_{p'}$

Interaction between ABD and Multi-Paxos. In Pineapple, ABD handles reads and writes, whereas Multi-Paxos takes care of RMWs. These two components use pstamps to inter-

act with each other – both use pstamps to determine which operation is more recent.

ABD runs as usual by using the tag part in pstamps. Writes only updates the tag part, whereas reads need to use the ordering of pstamps to decide which value to return. The value returned could be a value generated by a write or a RMW.

Multi-Paxos in Pineapple needs to do one more round of communication, compared to the original Multi-Paxos. This is because the leader might not be in the quorum for an ABD write. More concretely, the leader performs the following steps to complete a RMW with $f(\cdot)$:

- Contact a quorum Q , which includes the leader itself, to retrieve the value $v^{\max}$ that has the largest pstamp $ps^{\max}$;
- Apply RMW by computing $f(v^{\max})$. Let the output be v^{new} ; and
- Compute new pstamp ps^{new} as $(ps^{\max}.tag, slot_l + 1)$.
- Transmit $(v^{\text{new}}, ps^{\text{new}})$ to all and wait for an acknowledgement from a quorum.

The comparison of pstamps is defined in §3.1. Moreover, the quorum Q must include the leader itself, as in the common case with a stable leader, the leader has the most recent information about prior RMWs. This is also the reason that in the third step, the leader can update the pstamp by incrementing the local slot index $slot_l$ directly.

In the case when leader fails or changes, the leader N_l will use the meta-data of Multi-Paxos, e.g., ballots (not presented for conciseness), to detect such a scenario. Then Pineapple will need to participate in Phase 1 (the prepare phase) of Multi-Paxos to figure out the next leader. In this period, some operations might not be able to make progress. Same as other consensus algorithms, e.g., Multi-Paxos and Raft, Pineapple requires partial synchrony to elect a new stable leader and ensure liveness. Appendix A presents more details.

Furthermore, ABD reads in Algorithm 1 needs to be modified slightly so that they do not return inconsistent results when a leader change. Appendix A.2 discusses the extension.

Pineapple: Pseudo-code and Correctness. Algorithm 1 presents the ABD component of Pineapple. The logic is iden-

³In the full version of Pineapple, the slot index also needs to include necessary meta-data for Multi-Paxos, e.g., the ballot and leader ID. Such information is needed for handling leader changes. To keep our discussion concise, the second field is a single integer.

tical to the one described in §2.2, except that Pineapple uses pstamps except for tags. For each key key , $Storage_i[key]$ keeps two fields:

- $Storage_i[key].value$: the current value for key key , and
- $Storage_i[key].ps$: the associated pstamp.

Pineapple also adopts various optimizations mentioned in [10, 25, 42]. As a result, Pineapple is able to always complete reads in 1 RTT when $n = 3$.

Algorithm 2 presents the Multi-Paxos component, assuming that there is a stable leader N_l . Compared to the original Multi-Paxos, Pineapple needs to do the Get phase to learn the most recent value for the $K_{input} = \{key\}$. The Put phase is basically the same as Phase 2 (the propose phase) in Multi-Paxos, which updates a quorum (of acceptors) with the value generated by the RMW.

Analysis. Since both ABD and Multi-Paxos use a majority quorum, the quorum size is $\lfloor n/2 \rfloor + 1$. As in prior optimizations, ABD can complete on the fast path when the quorum returns the same response. RMW takes 3-RTT if the client is not co-located with the leader. All the operations have non-blocking execution, because they can be executed upon learning a decision either from a quorum or a leader.

Correctness. On a high-level, the correctness follows from three observations: (i) the correctness of ABD to order reads and writes; (ii) the correctness of Multi-Paxos to order RMWs; and (iii) the usage of pstamps to ensure that if a RMW o is visible to any read, then any future operation must also observe o or any operation that has pstamps larger than o 's. Appendix B presents the full proof.

We use examples from §2 to provide an intuition on the correctness. First, RMWs go through the leader; hence the ordering is stable. In particular, if the leader sees o_8 when applying RMW o_{24} in the Get phase (Line 1 in Algorithm 2), then other nodes will order o_8 before o_{24} . The reasons that this ordering is stable are: (i) the leader decides the order, which holds true by the design of Multi-Paxos, even when there is a leader change; and (ii) the pstamps correctly capture the fact that o_{24} happens after o_8 – which is due to the design that the leader increments slot index (Line 6 in Algorithm 2).

Second, multi-key operations, such as scans and MSETs, are also ordered by the leader. Therefore, inconsistency (or incomparability) in Figure 1 is impossible. This is intuitively because that all nodes will observe the sequence (of the slots) following the stable order determined by the leader.

In Figure 1, both scans o_3 and o_4 are served by the leader N_l . Suppose N_l observes o_1 , but not o_2 in the Get phase, so it returns $(1, \perp)$ for o_3 . When serving o_4 , N_l can only return $(1, \perp)$ or $(1, 2)$. This is because the quorum Q at Line 3 in Algorithm 2 must contain N_l itself. In other words, $(\perp, 2)$, an incomparable result in the example from §2, can never be returned by N_l , as it has already observed o_3 .

Finally, the reason that writes can be ordered in a non-stable manner is because of the tag part of the pstamps and

its semantic. If a node i receives a write with a pstamp larger than its local value, then i can safely overwrite its local value; otherwise, the write can safely be discarded. Furthermore, even though the ordering of concurrent writes is non-stable in general, whenever the leader observes them, the relative ordering between writes and RMWs becomes stable.

More concretely, consider a write w concurrent with a RMW r . We have two cases:

Case I: If the leader observes w during the Get phase, then r must have a larger pstamp, which allows all nodes to ignore w , after applying r . In other words, w *cannot* overwrite r .

Case II: If the leader does not observe w , then there are two subcases: (i) w 's pstamp is larger than o 's, or (ii) there is another write w' whose pstamp is larger than w and has the same tag part as in r 's pstamp.

- In (i), the result of r will be overwritten by w . (The total ordering is r, w .)
- In (ii), w will be discarded by nodes that have applied r . (The total ordering is w, w', r)

In both subcases, linearizability and the semantics are satisfied, because the leader's observation in the Get phase makes the relative ordering among w, w' and r stable.

4 Evaluation

We aim to compare Pineapple with other systems that have similar goals and target general environments that are appropriate for Multi-Paxos. In other words, we target systems that do not make specific assumptions on the hardware or networks. Recall that §1 mentions four categories of solutions that can be used to improve consensus-based linearizable storages. The purpose of our evaluation is focused on category (iv) – understanding the performance of new algorithmic components that are integrated with existing consensus algorithms, since systems in other categories typically adopt different architectures and/or assumptions (more discussion in §5).

We compare Pineapple with five other systems – Multi-Paxos, Multi-Paxos with leader lease, PQR [13], EPaxos [46], and Gryff [10]. We evaluate throughput and latency in both local-area network (LAN) and wide-area network (WAN), with the focus on geo-replicated, large-scale web applications. For these evaluations, we focus on tail latency, because user-perceived latency in these applications is highly correlated with the tail latency of the underlying storage systems [7, 19, 52, 58]. Finally, we compare Pineapple with the optimized Raft in a production system etc.

4.1 Implementation and Experiment Setup

Implementation. We implement Pineapple, PQR [13], and Multi-Paxos with leader lease in Go using the framework of EPaxos, Gryff and Multi-Paxos from [10, 46] to ensure a fair comparison. In the discussion below, we will use “Paxos

	CA	VA	IR	OR	JP
CA	0.2				
VA	72	0.2			
IR	151	88	0.2		
OR	59	93	145	0.2	
JP	113	162	220	121	0.2

Table 2: **RTT (in ms) between VMs in emulated geographic regions** [10]. For $n = 3$, we use region CA, VA, and IR.

systems” to refer to Multi-Paxos, Multi-Paxos with lease and PQR, as they have similar performance pattern in most cases.

Leases [27, 36, 65, 67, 73] are practical optimizations to reduce communication overhead, by requiring *additional assumptions* on timing (or synchrony). We implement the version of leader lease from [66, 67], which assumes a known bound on clock drift so that only one node holds a lease at a time. In the implementation, the leader can serve reads *without* communicating with another node, while holding the lease. Since we focus on the common case, we assume that the leader has the lease at all time during our evaluation.

We do *not* enable thrifty optimization *nor* batching, because these optimizations generally increase tail latency due to a higher chance of conflicts [10, 46, 61]. Thrifty operations send some messages to a necessary quorum only; however, it makes EPaxos easier to build a long chain of dependency, and makes nodes wait longer before an operation can be executed. Since we are targeting large-scale customer-facing applications, we focus on configurations that favor low tail latency.

We also implement Pineapple in etcd by replacing the Raft algorithm. This allows us to study how Pineapple interacts with the other layers of a production key-value system. In addition, etcd persists the states to disk; hence, etcd experiments also allow us to investigate the impacts of persistence.

Testbed. We run our experiments on CloudLab [21] using c6525-25g (AMD 7302P at 3.00GHz, 16 cores, 128 GB RAM) for both node and client VMs. For LAN, the ping latency is around 0.15ms and the configured link bandwidth is 10 Gbps. For WAN, we adopt the same latency profile used in [10, 63] – (i) $n = 3$: nodes in California (CA), Virginia (VA), and Ireland (IR); and (ii) $n = 5$: two more nodes in Oregon (OR) and Japan (JP). The WAN latencies are emulated using Linux’s Traffic Control (tc) by adding delays to packets on all VMs with filters on different IPs. Table 2 shows the configured RTT between nodes and clients in different regions. These numbers were chosen to represent typical RTT between the corresponding Amazon EC2 availability regions [10, 63].

Experiment Setup. The systems are configured to commit and execute the operations before responding to client requests (close-loop clients). Since as observed in [10, 61, 63], most web applications depend on the confirmation returned by an operation. For example, both Redis [2] and ZooKeeper [32] return results for both reads and writes.

Each experiment is run for 180 seconds, and we collect statistics in the middle of 150 seconds. The statistics collected in this period are quite stable, after the removal of fluctuating numbers during warm-up and cool-down. Following prior setups [10, 46, 61, 63], each object is of 16B. This is because 16B gives the best performance for EPaxos and Gryff. Furthermore, this also reflects real-world observation. For example, the report from Facebook [7] suggests that 40% of the data in their Memcached workload is less than 11B. In Facebook’s production TAO system [9, 52], 50% of its requests have the value field smaller than 16B. When evaluating Pineapple in etcd, we test it with a larger object size at 100B.

Following [10, 13, 46, 61, 63], all systems we test store data in memory, except for the etcd experiments. This strategy is reliable as long as the number of *concurrent* node failures is bounded, which is a common case with $n = 2f + 1$ [15, 17, 24]. Moreover, recent hardware solutions are being developed and adopted to prevent data loss from crashed machines, e.g., persistent memory or disaggregated memory [18, 62, 70, 72].

Two operations are said to be “*conflicting*” with each other if they are targeting the same object (or same key) [10, 15, 46, 61, 63]. A conflict rate p means that (i) with a probability p , a client chooses the same key that is defined in advance, and (ii) with probability $1 - p$, a client chooses a unique key.

Our evaluation does not use Zipfian distribution [16, 61], except when using YCSB for etcd experiments, because we want to pick favorable configurations for EPaxos and Gryff. Due to their execution logic, they suffer performance degrade under Zipfian workloads [10, 61].

4.2 Summary of Our Findings

We aim at answering the following questions:

- Does Pineapple improve maximum attainable throughput in both LAN and WAN? (§4.3)
- How do the read/write/RMW latency of Pineapple compare against state-of-the-art systems? (§4.4)
- How does Pineapple perform as the consensus layer in a production system etcd? (§4.5)

To summarize our findings,

- In LAN, Pineapple improves throughput by 10% – 20%, over the closest competitor (EPaxos) when the RMW ratio is less than 20%. In WAN, Pineapple has around 3.5x throughput, over the closest competitor (PQR and Multi-Paxos with lease) in a balanced workload.
- In WAN, Pineapple generally has better read tail latency than Paxos systems in 3- and 5-node deployments under web application workloads [10, 16]. Pineapple has better RMW tail latency than Gryff and EPaxos in a 3-node deployment under workloads with 25% conflict rate.
- We compare the performance of Pineapple and an optimized Raft in etcd [1]. Our evaluation using YCSB [16] shows that Pineapple reduces median latency by 20% to 50% in a balanced workload in LAN.

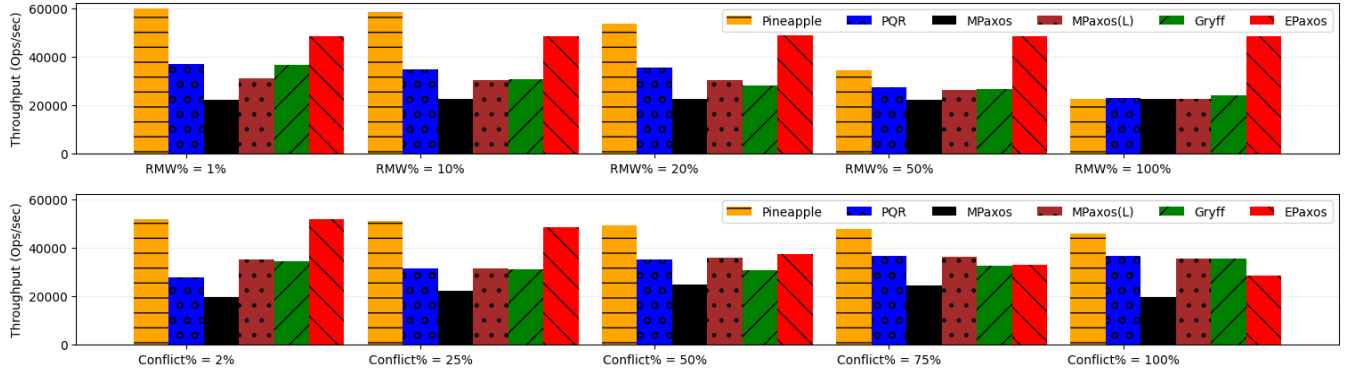


Figure 2: **Maximum attainable throughput in 5-node LAN** ($n = 5$, 49.5% reads, 49.5% writes). Top row varies RMW ratios with conflict rate 25%. Bottom row varies conflict rates with RMW ratio = 20%. MPaxos is the label for Multi-Paxos, whereas MPaxos(L) is the label for Multi-Paxos with leader lease.

4.3 Maximum Attainable Throughput

To achieve maximum throughput, we load each system up to saturation and we collect the median throughput during the period around saturation.

Throughput in LAN. Figure 2 shows the maximum throughput with 1% to 100% RMWs ratio. The remaining operations consist of an equal number of reads and writes. This reflects the balanced workload of YCSB-A [16]. In addition, the numbers help estimate the impact of RMWs and the bottleneck of the leader in Paxos systems and Pineapple.

In this experiment, $n = 5$. Each system requires different number of close-loop clients to saturate (ranging from 150 to 300 clients). In Pineapple, PQR, EPaxos, and Gryff, the clients are evenly distributed so that each node handles the same number of clients. In other two systems, the clients only communicate with the leader node. In PQR and Pineapple, some clients need to communicate with both the leader and one follower node. Top row has conflict rate = 25% so that we capture the effect of conflicting operations.

With RMW ratio $\leq 20\%$, Pineapple has 10% – 20% improvement over EPaxos, the closest competitor. When RMW ratio is 100%, Pineapple is slightly slower than Paxos systems, because RMWs go through the leader and Pineapple’s leader needs to process more messages for RMWs.

EPaxos, on the other hand, remains stable throughout different ratios. It has best performance with higher RMW ratios, at around 2.2x over other systems. Compared to Pineapple and Paxos systems, EPaxos’s leaderless design improves throughput. Compared to Gryff, it has simpler execution logic.

Bottom row demonstrates that all systems under test, except for EPaxos, have stable throughput under different conflict rates. This is because with higher conflict rates, EPaxos’s throughput is affected by its growing dependency graph. Gryff is less affected because only RMWs need to resolve dependency. Multi-Paxos has the worst throughput, across all conflict rates, because of the saturated leader.

Throughput in WAN. In WAN, Pineapple has 4x to 10x throughput, compared to EPaxos, in read-heavy workloads (a variant of YCSB-B), and around 3.5x throughput in balanced workloads (a variant of YCSB-A). This is because as shown in [10, 61], EPaxos’s dependency graph is easier to grow in size in WAN, which significantly affects its throughput. Due to page limits, we present details in Appendix C.

In read-heavy workloads, “Multi-Paxos with lease” has best performance, with 1.25x to 1.3x throughput, compared to Pineapple’s. This is because with lease, the leader does not need to process as many messages as Pineapple does. In balanced workloads, Pineapple has best performance, which has 3x to 4x throughput improvement over the next closest competitor (PQR and “Multi-Paxos with lease”). This is because the leader in Paxos systems is saturated by writes and RMWs, whereas writes are evenly distributed in Pineapple.

4.4 Latency

4.4.1 Latency-Throughput in LAN

Figure 3 plots the throughput against median (p50) and 90-percentile (p90) latency for all operations by increasing the number of concurrent closed-loop clients (3 to 360) in LAN with $n = 5$. We use a variation of YCSB-A balanced workload with 1% RMWs from [10] and 25% conflicts to measure the impact of RMWs and concurrent operations. Pineapple has around 20% improvement in throughput at saturation. Pineapple in general has smallest p50 and p90 latency across all loads. Gryff has low latency but has limited throughput. This demonstrates that compared to Gryff, the usage of Multi-Paxos in Pineapple effectively improves performance

4.4.2 Latency in WAN

Read-heavy Workload with $n = 3$. Same as in [10], we use a variant of YCSB-B [16] workload that contains 94.5% reads,

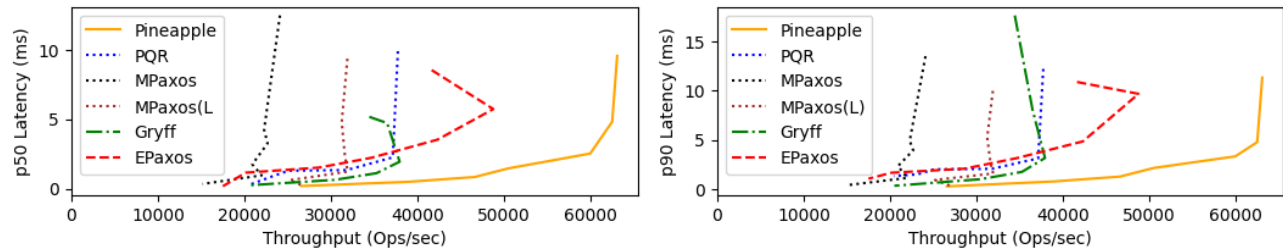


Figure 3: **Latency vs. Throughput in LAN** ($n = 5$, 49.5% reads, 49.5% writes, 1% RMWs, 25% conflict rate). Left figure presents median (p50) latency, whereas right figure presents 90-percentile (p90) latency.

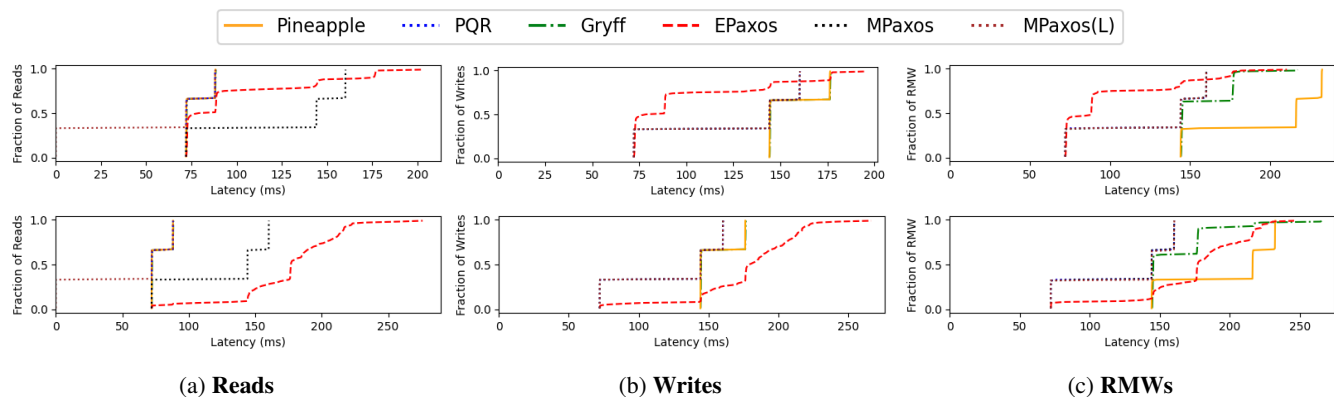


Figure 4: **Latency CDF** ($n = 3$, 94.5% reads, 4.5% writes, 1% RMWs). Top row has 2% conflict rate, whereas bottom row has 25%. Both Gryff and Pineapple complete reads in 1 RTT when $n = 3$. Leader of Paxos systems and Pineapple is in VA region.

4.5% writes, and 1.0% RMWs. The focus is on a read-heavy workload to reflect the pattern of web applications [9, 10, 16, 17]. In this set of experiments, the leader node is located in the VA region (c.f. Table 2). This setup is favorable to Paxos systems, as the leader is in the *nearest* quorum for both CA and IR. Following [10], each region has 16 concurrent clients. Figure 4 shows the latency CDF (cumulative distribution function) of all three operations with $n = 3$.

1-RTT Reads in Gryff and Pineapple. For $n = 3$, both Gryff and Pineapple always complete reads in 1 RTT. Figure 4a shows that an operation is completed after nodes spend 1 RTT to the nearest quorum, regardless of the conflict rate.

As in [10, 46, 61], clients and nodes are co-located; hence, RTTs between them is negligible. Clients in VA are closest to the nodes in VA and CA and vice versa for clients in CA. This results in around 66% of the reads completing in the RTT between VA and CA (72 ms). Clients in IR take 88 ms – these are the tail latency for reads.

Stable Performance in Paxos Systems. The leader is in the VA region; hence, the blocking execution of PQR’s reads is negligible. This is because the leader is always in the closest quorum. For both writes and RMWs, PQR takes 1 to 2 RTTs to complete. The tail latency is at 160 ms, which occurs when the clients in IR contact the leader in VA (88 ms), which then contacts the nearest quorum (72 ms). 1/3 of the reads in

“Multi-Paxos with lease” have latency close to 0, because the leader can serve them immediately.

Tail latency of RMWs. Pineapple’s RMWs take 3 RTT at follower nodes. Thus, the tail latency in Figure 4c is $72 * 2 + 88 = 232$, where the first part is the time from VA to the nearest quorum (CA) and the second part is the time the IR clients take for contacting the leader in VA.

With 2% conflicts and low percentage of non-read operations, Gryff has relatively stable RMW performance. With 25% conflicts, Pineapple has roughly 30 ms improvement at p99 latency relative to Gryff, because the execution of RMWs in Gryff is blocking. EPaxos in general has worse tail latency, due to its dependency tracking.

Blocking Operation Execution. Figure 5 presents the latency CDF for reads and writes under the same workload. To better demonstrate the blocking effect, the leaders are located in CA. PQR’s reads cannot complete in 1 RTT, because the leader is not in IR’s closest quorum. As a result, the tail latency of PQR is around 70 ms longer than Gryff’s and Pineapple’s.

Gryff’s RMWs has longer tail latency under higher conflicts, because of longer dependency. In this case, due to Pineapple’s 3-RTT RMWs, its tail latency is still the largest under low workload. Figure 6 uses exactly the same configuration as in the bottom row of Figure 5c, except with a higher number of clients at 250 clients per region. More concurrent

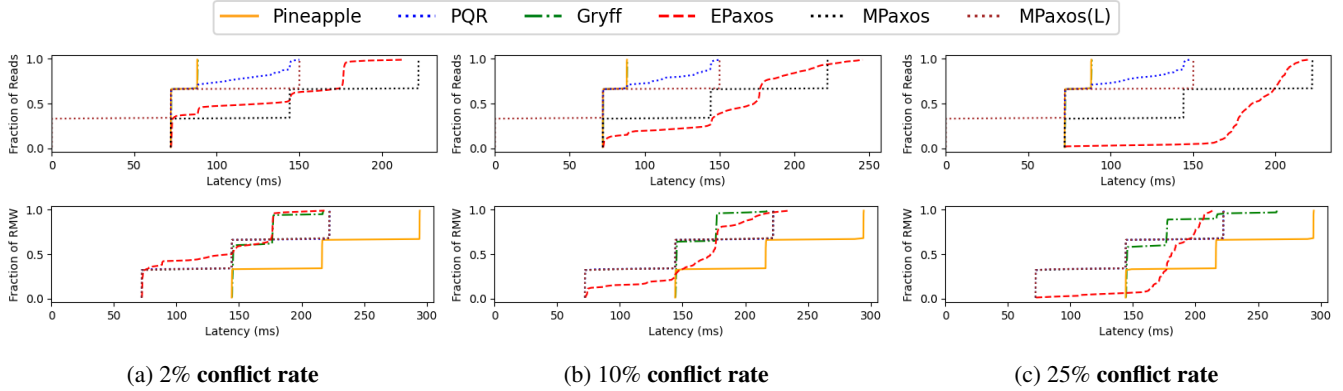


Figure 5: **Latency CDF** ($n = 3$, 94.5% reads, 4.5% writes, 1% RMWs). Top row plots the latency CDF for reads, whereas bottom row plots the latency CDF for RMWs. Leader of Paxos systems and Pineapple is in CA region.

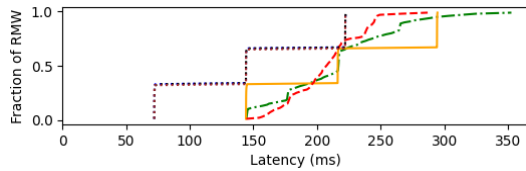


Figure 6: **Latency CDF** ($n = 3$, 94.5% reads, 4.5% writes, 1% RMWs, 250 clients)

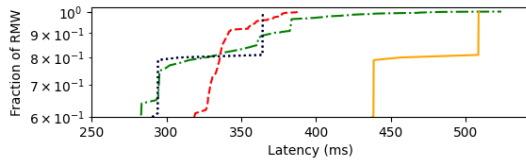


Figure 7: **Log-scale Latency CDF** ($n = 5$, 49.5% reads, 49.5% writes, 1% RMWs, 25% conflict rate, 120 clients).

operations lead to longer RMW tail latency in Gryff.

Balanced Workload with $n = 5$. We next use the same variant of YCSB-A from [10], which consists of 49.5% reads, 49.5% writes, and 1.0% RMWs with 2% and 25% conflicts. The balance between reads and writes and a higher conflict rate allow us to quantify the impacts of interleaving operations on latency. Figure 8 shows latency CDF for each operation type for $n = 5$. The leader is in the IR region.

1-RTT Reads. For $n = 5$, Gryff often completes reads in 1 RTT with 2% conflict rate (Figure 8a). Pineapple has slightly longer read tail latency, because Pineapple’s slower RMWs increase the chance of concurrent operations. When conflict is low, Pineapple completes reads faster than Paxos systems.

2-RTT Writes. As indicated in Table 1, in the worst case, all systems have 2-RTT writes; however, the latency pattern is different as it depends on where the clients are located (Figure 8b). For Gryff and Pineapple, the furthest site is IR; hence, 2 RTTs to the nearest quorum take $2 * 145 = 290$ ms.

In comparison, in Paxos systems, the tail latency for JP clients is $145 + 220 = 365$ ms.

Trade-offs. Pineapple has slower RMWs but faster writes and reads than Paxos Systems and EPaxos. It has longer read tail latency than “Multi-Paxos with lease” and PQR with 25% conflicts. Gryff has best latency in most cases under low workload. As a trade-off, Pineapple has better throughput. In these balanced workloads, Pineapple has 3x and 4x throughput, compared to closest competitors (c.f. Appendix C).

Figure 7 uses an identical configuration as in Figure 8, except with 120 clients per region. Under 25% conflicts, Gryff in fact has longer RMW tail latency than Pineapple.

4.5 Performance in Real-World System

We evaluate how Pineapple compares to optimized implementation of Raft in a production key-value storage designed for web applications. We replace the consensus layer in etcd by Pineapple. These experiments use a 5-node etcd deployment with YCSB client (Zipfian workloads with default key-value size 100KB). Each configuration runs roughly 1 to 2 minutes.⁴ YCSB’s threadcount ranges from 2 to 90. In our experience, this number allows etcd to have the most stable performance.

Figure 9a plots the throughput against p50 and p90 latency for all operations under YCSB-C (100% reads). Raft in etcd adopts the heartbeat-based batching optimization for improving read latency; hence, we use YCSB-C to see how Pineapple compares to a practical optimization. Figure 9b uses a variant of YCSB-A workload with 50% reads, 25% writes/RMWs, which allows us to see the impact of Pineapple’s 3-RTT RMWs. Figure 9c uses a variant of YCSB-B workload with 95% reads and 2.5% writes/RMWs.

Figure 9a shows that before saturation, Pineapple has around 20% improvement in both p50 and p90 latency in all-read workload. Figure 9b demonstrates that before saturation, Pineapple has 50% improvement in p50 latency. Pineapple

⁴YCSB only allows the configuration on number of operations; hence, the experiment time varies.

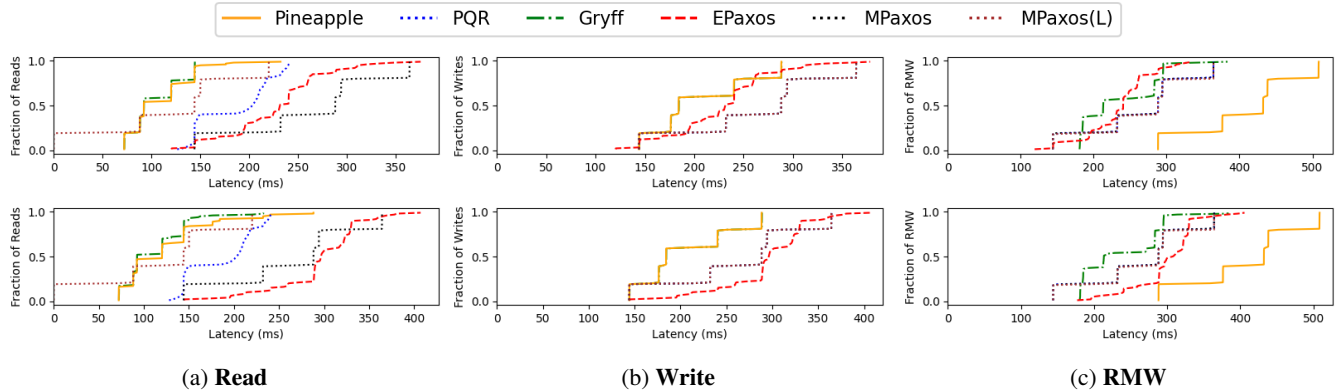


Figure 8: **Latency CDF** ($n = 5$, 49.5% reads, 49.5% writes, 1%RMW). Top row has 2% conflict rate, whereas bottom row has 25% conflict rate. Leader of Paxos systems and Pineapple is in IR region.

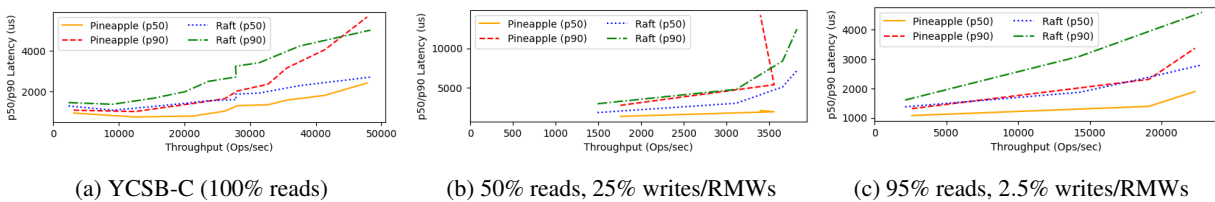


Figure 9: **Latency vs. Throughput in etcd**. ($n = 5$, Zipfian workloads)

has higher p90 latency and lower throughput mainly because of the high RMW ratio. However, Pineapple's p90 latency is within 10% and throughput within 15% of Raft. This indicates that Pineapple's RMWs has mild performance impact, when interacting with other components in a real-world system. Figure 9c shows that under read-heavy workload, Pineapple has a similar performance pattern, with 40% to 50% reduction in both p50 and p90 latency. In conclusion, offloading operations effectively reduces latency, even compared to an optimized implementation of Raft.

In these experiments, Pineapple's throughput is not higher, even when we observe the effect of saturated leader. This is mainly because Raft's read optimization enables an efficient disk batching (which is not implemented in Pineapple).

5 Related Work

Consensus is useful for making a service fault-tolerant [59]. Classical consensus uses a single leader, e.g., [33, 36, 39, 53]. Recent systems adopt a multi-leader design that distributes the ordering responsibility evenly to increase performance (e.g., Mencius [44], EPaxos [46], M²Paxos [55], Caesar [6], ATLAS [24]). The multi-leader design uses a graph for keeping track of dependency, which often leads to long tail latency [57].

Recent systems make improvements by utilizing modern assumptions or hardware. For example, Copilot is focused on tolerating slow leader [51], Rabia on stable network [54], Mu [5] on RDMA, Speculative Paxos [56] and NOPaxos [40] on SDN, and Tempo [23] considers a small number of con-

current failures. Other systems focus on scenarios different from Pineapple, e.g., Giza [15], Pando [64] and CRaft [68] are focused on erasure coded storages. Nil-externality [26] utilizes operations that modify state within a storage but has no externalized effects. Clock-RSM [20], Caesar [6], EPaxos-TOQ [61] rely on loosely synchronized clock

Another category of systems target communication. For example, read leases allow reads to complete without any inter-replica communication [28, 47]. This design generally leads to slower writes. Compartmentalized Paxos [69] adopts the idea of proxy nodes to bypass various communication and computation bottlenecks. PigPaxos [14] uses piggybacking to reduce communication. These systems potentially require more resources or a better network infrastructure.

All these systems could potentially achieve better performance than Pineapple, under a right set of assumptions. Our focus is on general deployments targeted by Multi-Paxos and Raft, including both LAN and WAN, *without* specific assumptions on the environment or modern hardware.

Transactional systems, such as Janus [49], Calvin [60], and TAPIR [71], provide richer semantics compared to Pineapple. In fact, Pineapple and Gryff are specifically designed and optimized for key-value storages with more restrictive operation semantics. Consequently, general transactional systems are facing more bottlenecks when serving single-key reads/writes and could have longer latency in general. For example, Janus requires the leader to resolve conflicting transactions, and Janus's use of dependency tracking (for reordering conflicting transactions) could also result in longer tail latency.

References

- [1] etcd: A distributed, reliable key-value store for the most critical data of a distributed system. <https://etcd.io/>, accessed May 2021.
- [2] RedisRaft: Strongly-consistent Redis deployments. <https://github.com/RedisLabs/redisraft>, accessed December 2020.
- [3] Replication Layer | CockroachDB Docs. <https://www.cockroachlabs.com/docs/stable/architecture/replication-layer.html>, accessed December 2020.
- [4] RethinkDB 2.1: high availability. <https://rethinkdb.com/blog/2.1-release/>, accessed December 2020.
- [5] Marcos K. Aguilera, Naama Ben-David, Rachid Guerraoui, Virendra J. Marathe, Athanasios Xygkis, and Igor Zablotchi. Microsecond consensus for microsecond applications. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*, pages 599–616, 2020.
- [6] Balaji Arun, Sebastiano Peluso, Roberto Palmieri, Giuliano Losa, and Binoy Ravindran. Speeding up consensus by chasing fast decisions. In *47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2017, Denver, CO, USA, June 26-29, 2017*, pages 49–60. IEEE Computer Society, 2017.
- [7] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. Workload analysis of a large-scale key-value store. In Peter G. Harrison, Martin F. Arlitt, and Giuliano Casale, editors, *ACM SIGMETRICS/PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS '12, London, United Kingdom, June 11-15, 2012*, pages 53–64. ACM, 2012.
- [8] Hagit Attiya, Amotz Bar-Noy, and Danny Dolev. Sharing memory robustly in message-passing systems. *J. ACM*, 42(1):124–142, January 1995.
- [9] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry C. Li, Mark Marchukov, Dmitri Petrov, Lovro Puzar, Yee Jiun Song, and Venkateshwaran Venkataramani. TAO: facebook’s distributed data store for the social graph. In Andrew Birrell and Emin Gün Sirer, editors, *2013 USENIX Annual Technical Conference, San Jose, CA, USA, June 26-28, 2013*, pages 49–60. USENIX Association, 2013.
- [10] Matthew Burke, Audrey Cheng, and Wyatt Lloyd. Gryff: Unifying consensus and shared registers. In Ranjita Bhagwan and George Porter, editors, *17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020, Santa Clara, CA, USA, February 25-27, 2020*, pages 591–617. USENIX Association, 2020.
- [11] Brad Calder, Ju Wang, Aaron Ogus, Niranjan Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, Jaidev Haridas, Chakravarthy Uddaraju, Hemal Khatri, Andrew Edwards, Vaman Bedekar, Shane Mainali, Rafay Abbasi, Arpit Agarwal, Mian Fahim ul Haq, Muhammad Ikram ul Haq, Deepali Bhardwaj, Sowmya Dayanand, Anitha Adusumilli, Marvin McNett, Sriram Sankaran, Kavitha Manivannan, and Leonidas Rigas. Windows azure storage: A highly available cloud storage service with strong consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles, SOSP '11*, pages 143–157, New York, NY, USA, 2011. Association for Computing Machinery.
- [12] Wei Cao, Yusong Gao, Feifei Li, Sheng Wang, Bingchen Lin, Ke Xu, Xiaojie Feng, Yucong Wang, Zhenjun Liu, and Gejin Zhang. Timon: A timestamped event database for efficient telemetry data processing and analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, SIGMOD '20*, page 739–753, New York, NY, USA, 2020. Association for Computing Machinery.
- [13] Aleksey Charapko, Ailidani Ailijiang, and Murat Demirbas. Linearizable quorum reads in paxos. In Daniel Peek and Gala Yadgar, editors, *11th USENIX Workshop on Hot Topics in Storage and File Systems, HotStorage 2019, Renton, WA, USA, July 8-9, 2019*. USENIX Association, 2019.
- [14] Aleksey Charapko, Ailidani Ailijiang, and Murat Demirbas. Pigpaxos: Devouring the communication bottlenecks in distributed consensus. In Guoliang Li, Zhanhuai Li, Stratos Idreos, and Divesh Srivastava, editors, *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, pages 235–247. ACM, 2021.
- [15] Yu Lin Chen, Shuai Mu, Jinyang Li, Cheng Huang, Jin Li, Aaron Ogus, and Douglas Phillips. Giza: Erasure coding objects across global data centers. In Dilma Da Silva and Bryan Ford, editors, *2017 USENIX Annual Technical Conference, USENIX ATC 2017, Santa Clara, CA, USA, July 12-14, 2017*, pages 539–551. USENIX Association, 2017.
- [16] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In Joseph M. Hellerstein,

Surajit Chaudhuri, and Mendel Rosenblum, editors, *Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC 2010, Indianapolis, Indiana, USA, June 10-11, 2010*, pages 143–154. ACM, 2010.

- [17] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson C. Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. Spanner: Google’s globally-distributed database. In Chandu Thekkath and Amin Vahdat, editors, *10th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2012, Hollywood, CA, USA, October 8-10, 2012*, pages 251–264. USENIX Association, 2012.
- [18] Intel Corporation. Intel optane persistent memory. <https://www.intel.com/content/www/us/en/architecture-and-technology/optane-dc-persistent-memory.html>, 2021.
- [19] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Commun. ACM*, 56(2):74–80, 2013.
- [20] Jiaqing Du, Daniele Sciascia, Sameh Elnikety, Willy Zwaenepoel, and Fernando Pedone. Clock-rsm: Low-latency inter-datacenter state machine replication using loosely synchronized physical clocks. In *44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2014, Atlanta, GA, USA, June 23-26, 2014*, pages 343–354. IEEE Computer Society, 2014.
- [21] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The design and operation of CloudLab. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pages 1–14, July 2019.
- [22] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. Consensus in the presence of partial synchrony. *J. ACM*, 35(2):288–323, 1988.
- [23] Vitor Enes, Carlos Baquero, Alexey Gotsman, and Pierre Sutra. Efficient replication via timestamp stability. In Antonio Barbalace, Pramod Bhatotia, Lorenzo Alvisi, and Cristian Cadar, editors, *EuroSys ’21: Sixteenth European Conference on Computer Systems, Online Event, United Kingdom, April 26-28, 2021*, pages 178–193. ACM, 2021.
- [24] Vitor Enes, Carlos Baquero, Tuanir França Rezende, Alexey Gotsman, Matthieu Perrin, and Pierre Sutra. State-machine replication for planet-scale systems. In Angelos Bilas, Kostas Magoutis, Evangelos P. Markatos, Dejan Kostic, and Margo I. Seltzer, editors, *EuroSys ’20: Fifteenth EuroSys Conference 2020, Heraklion, Greece, April 27-30, 2020*, pages 24:1–24:15. ACM, 2020.
- [25] Burkhard Englert, Chryssis Georgiou, Peter M. Muesal, Nicolas C. Nicolaou, and Alexander A. Shvartsman. On the efficiency of atomic multi-reader, multi-writer distributed memory. In Tarek F. Abdelzaher, Michel Raynal, and Nicola Santoro, editors, *Principles of Distributed Systems, 13th International Conference, OPODIS 2009, Nîmes, France, December 15-18, 2009. Proceedings*, volume 5923 of *Lecture Notes in Computer Science*, pages 240–254. Springer, 2009.
- [26] Aishwarya Ganesan, Ramnathan Alagappan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Exploiting nil-externality for fast replicated storage. In Robbert van Renesse and Nikolai Zeldovich, editors, *SOSP ’21: ACM SIGOPS 28th Symposium on Operating Systems Principles, Virtual Event / Koblenz, Germany, October 26-29, 2021*, pages 440–456. ACM, 2021.
- [27] C. Gray and D. Cheriton. Leases: an efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles, SOSP ’89*, page 202–210, New York, NY, USA, 1989. Association for Computing Machinery.
- [28] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In Gregory R. Andrews, editor, *Proceedings of the Twelfth ACM Symposium on Operating System Principles, SOSP 1989, The Wigwam, Litchfield Park, Arizona, USA, December 3-6, 1989*, pages 202–210. ACM, 1989.
- [29] Maurice Herlihy. Wait-free synchronization. *ACM Trans. Program. Lang. Syst.*, 13(1):124–149, 1991.
- [30] Maurice P Herlihy and Jeannette M Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 12(3):463–492, 1990.
- [31] Heidi Howard, Dahlia Malkhi, and Alexander Spiegelman. Flexible paxos: Quorum intersection revisited. In Panagioti Fatourou, Ernesto Jiménez, and Fernando Pedone, editors, *20th International Conference on Principles of Distributed Systems, OPODIS 2016, December*

- 13-16, 2016, Madrid, Spain, volume 70 of *LIPICs*, pages 25:1–25:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016.
- [32] Patrick Hunt, Mahadev Konar, Flavio Paiva Junqueira, and Benjamin Reed. Zookeeper: Wait-free coordination for internet-scale systems. In Paul Barham and Timothy Roscoe, editors, *2010 USENIX Annual Technical Conference, Boston, MA, USA, June 23-25, 2010*. USENIX Association, 2010.
 - [33] Flavio P. Junqueira, Benjamin C. Reed, and Marco Serafini. Zab: High-performance broadcast for primary-backup systems. In *Proceedings of the 2011 IEEE/IFIP 41st International Conference on Dependable Systems and Networks, DSN '11*, page 245–256, USA, 2011. IEEE Computer Society.
 - [34] Robert Kallman, Hideaki Kimura, Jonathan Natkins, Andrew Pavlo, Alexander Rasin, Stanley Zdonik, Evan P. C. Jones, Samuel Madden, Michael Stonebraker, Yang Zhang, John Hugg, and Daniel J. Abadi. H-store: a high-performance, distributed main memory transaction processing system. *Proc. VLDB Endow.*, 1(2):1496–1499, August 2008.
 - [35] Avinash Lakshman and Prashant Malik. Cassandra: a decentralized structured storage system. *Operating Systems Review*, 44(2):35–40, 2010.
 - [36] Leslie Lamport. The part-time parliament. *ACM Transactions on Computer Systems (TOCS)*, 16(2):133–169, 1998.
 - [37] Leslie Lamport. Generalized consensus and paxos. Technical report, Technical Report MSR-TR-2005-33, Microsoft Research, 2005.
 - [38] Leslie Lamport. Fast paxos. *Distributed Computing*, 19(2):79–103, 2006.
 - [39] Leslie Lamport et al. Paxos made simple. *ACM Sigact News*, 32(4):18–25, 2001.
 - [40] Jialin Li, Ellis Michael, Naveen Kr. Sharma, Adriana Szekeres, and Dan R. K. Ports. Just say NO to paxos overhead: Replacing consensus with network ordering. In Kimberly Keeton and Timothy Roscoe, editors, *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*, pages 467–483. USENIX Association, 2016.
 - [41] Nancy A. Lynch. *Distributed Algorithms*. Morgan Kaufmann, 1996.
 - [42] Nancy A. Lynch and Alexander A. Shvartsman. Robust emulation of shared memory using dynamic quorum-acknowledged broadcasts. In *Digest of Papers: FTCS-27, The Twenty-Seventh Annual International Symposium on Fault-Tolerant Computing, Seattle, Washington, USA, June 24-27, 1997*, pages 272–281. IEEE Computer Society, 1997.
 - [43] Yanhua Mao, Flavio Paiva Junqueira, and Keith Marzullo. Mencius: Building efficient replicated state machine for wans. In Richard Draves and Robbert van Renesse, editors, *8th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2008, December 8-10, 2008, San Diego, California, USA, Proceedings*, pages 369–384. USENIX Association, 2008.
 - [44] Yanhua Mao, Flavio Paiva Junqueira, and Keith Marzullo. Mencius: Building efficient replicated state machine for wans. In Richard Draves and Robbert van Renesse, editors, *8th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2008, December 8-10, 2008, San Diego, California, USA, Proceedings*, pages 369–384. USENIX Association, 2008.
 - [45] Christopher Mitchell, Yifeng Geng, and Jinyang Li. Using one-sided rdma reads to build a fast, cpu-efficient key-value store. In *Proceedings of the 2013 USENIX Conference on Annual Technical Conference, USENIX ATC'13*, pages 103–114, USA, 2013. USENIX Association.
 - [46] Iulian Moraru, David G. Andersen, and Michael Kaminsky. There is more consensus in egalitarian parliaments. In Michael Kaminsky and Mike Dahlin, editors, *ACM SIGOPS 24th Symposium on Operating Systems Principles, SOSP '13, Farmington, PA, USA, November 3-6, 2013*, pages 358–372. ACM, 2013.
 - [47] Iulian Moraru, David G. Andersen, and Michael Kaminsky. Paxos quorum leases: Fast reads without sacrificing writes. In Ed Lazowska, Doug Terry, Remzi H. Arpaci-Dusseau, and Johannes Gehrke, editors, *Proceedings of the ACM Symposium on Cloud Computing, Seattle, WA, USA, November 3-5, 2014*, pages 22:1–22:13. ACM, 2014.
 - [48] Shuai Mu, Yang Cui, Yang Zhang, Wyatt Lloyd, and Jinyang Li. Extracting more concurrency from distributed transactions. In *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation, OSDI'14*, page 479–494, USA, 2014. USENIX Association.
 - [49] Shuai Mu, Lamont Nelson, Wyatt Lloyd, and Jinyang Li. Consolidating concurrency control and consensus for commits under conflicts. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI*

- 16), pages 517–532, Savannah, GA, November 2016. USENIX Association.
- [50] Khiem Ngo, Siddhartha Sen, and Wyatt Lloyd. Tolerating slowdowns in replicated state machines using copilots. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*, pages 583–598. USENIX Association, 2020.
 - [51] Khiem Ngo, Siddhartha Sen, and Wyatt Lloyd. Tolerating slowdowns in replicated state machines using copilots. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*, pages 583–598. USENIX Association, 2020.
 - [52] Rajesh Nishtala, Hans Fugal, Steven Grimm, Marc Kwiatkowski, Herman Lee, Harry C. Li, Ryan McElroy, Mike Paleczny, Daniel Peek, Paul Saab, David Stafford, Tony Tung, and Venkateshwaran Venkataramani. Scaling memcache at facebook. In Nick Feamster and Jeffrey C. Mogul, editors, *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2013, Lombard, IL, USA, April 2-5, 2013*, pages 385–398. USENIX Association, 2013.
 - [53] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference, USENIX ATC’14*, page 305–320, USA, 2014. USENIX Association.
 - [54] Haochen Pan, Jesse Tuglu, Neo Zhou, Tianshu Wang, Yicheng Shen, Xiong Zheng, Joseph Tassarotti, Lewis Tseng, and Roberto Palmieri. Rabia: Simplifying state-machine replication through randomization. In Robbert van Renesse and Nickolai Zeldovich, editors, *SOSP ’21: ACM SIGOPS 28th Symposium on Operating Systems Principles, Virtual Event / Koblenz, Germany, October 26-29, 2021*, pages 472–487. ACM, 2021.
 - [55] Sebastiano Peluso, Alexandru Turcu, Roberto Palmieri, Giuliano Losa, and Binoy Ravindran. Making fast consensus generally faster. In *46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2016, Toulouse, France, June 28 - July 1, 2016*, pages 156–167. IEEE Computer Society, 2016.
 - [56] Dan R. K. Ports, Jialin Li, Vincent Liu, Naveen Kr. Sharma, and Arvind Krishnamurthy. Designing distributed systems using approximate synchrony in data center networks. In *12th USENIX Symposium on Networked Systems Design and Implementation, NSDI 15, Oakland, CA, USA, May 4-6, 2015*, pages 43–57. USENIX Association, 2015.
 - [57] Tuanir França Rezende and Pierre Sutra. Leaderless state-machine replication: Specification, properties, limits. In Hagit Attiya, editor, *34th International Symposium on Distributed Computing, DISC 2020, October 12-16, 2020, Virtual Conference*, volume 179 of *LIPICs*, pages 24:1–24:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.
 - [58] High Scalability. Latency is everywhere and it costs you sales - how to crush it <http://highscalability.com/latency-everywhere-and-it-costs-you-sales-how-crush-it>, 2009, accessed 2021.
 - [59] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Comput. Surv.*, 22(4):299–319, 1990.
 - [60] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. Calvin: fast distributed transactions for partitioned database systems. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data, SIGMOD ’12*, page 1–12, New York, NY, USA, 2012. Association for Computing Machinery.
 - [61] Sarah Tollman, Seo Jin Park, and John K. Ousterhout. Epaxos revisited. In James Mickens and Renata Teixeira, editors, *18th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2021, April 12-14, 2021*, pages 613–632. USENIX Association, 2021.
 - [62] Shin-Yeh Tsai, Yizhou Shan, and Yiyang Zhang. Disaggregating persistent memory and controlling them remotely: An exploration of passive disaggregated key-value stores. In Ada Gavrilovska and Erez Zadok, editors, *2020 USENIX Annual Technical Conference, USENIX ATC 2020, July 15-17, 2020*, pages 33–48. USENIX Association, 2020.
 - [63] Lewis Tseng, Neo Zhou, Cole Dumas, Tigran Bantikyan, and Roberto Palmieri. Distributed multi-writer multi-reader atomic register with optimistically fast read and write. In Kunal Agrawal and Julian Shun, editors, *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures, SPAA 2023, Orlando, FL, USA, June 17-19, 2023*, pages 479–488. ACM, 2023.
 - [64] Muhammed Uluyol, Anthony Huang, Ayush Goel, Mosharaf Chowdhury, and Harsha V. Madhyastha. Near-optimal latency versus cost tradeoffs in geo-distributed storage. In Ranjita Bhagwan and George Porter, editors, *17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020, Santa Clara, CA, USA, February 25-27, 2020*, pages 157–180. USENIX Association, 2020.

- [65] Robbert Van Renesse and Deniz Altinbuken. Paxos made moderately complex. *ACM Comput. Surv.*, 47(3), feb 2015.
- [66] Robbert Van Renesse and Deniz Altinbuken. Paxos made moderately complex. *ACM Comput. Surv.*, 47(3), February 2015.
- [67] Robbert van Renesse, Fred B. Schneider, and Johannes Gehrke. Nerio: Leader election and edict ordering. *CoRR*, abs/1109.5111, 2011.
- [68] Zizhong Wang, Tongliang Li, Haixia Wang, Airan Shao, Yunren Bai, Shangming Cai, Zihan Xu, and Dongsheng Wang. Craft: An erasure-coding-supported version of raft for reducing storage cost and network cost. In Sam H. Noh and Brent Welch, editors, *18th USENIX Conference on File and Storage Technologies, FAST 2020, Santa Clara, CA, USA, February 24-27, 2020*, pages 297–308. USENIX Association, 2020.
- [69] Michael J. Whittaker, Ailidani Ailijiang, Aleksey Charapko, Murat Demirbas, Neil Giridharan, Joseph M. Hellerstein, Heidi Howard, Ion Stoica, and Adriana Szekeres. Scaling replicated state machines with compartmentalization. *Proc. VLDB Endow.*, 14(11):2203–2215, 2021.
- [70] Jian Yang, Juno Kim, Morteza Hoseinzadeh, Joseph Izraelevitz, and Steven Swanson. An empirical guide to the behavior and use of scalable persistent memory. In Sam H. Noh and Brent Welch, editors, *18th USENIX Conference on File and Storage Technologies, FAST 2020, Santa Clara, CA, USA, February 24-27, 2020*, pages 169–182. USENIX Association, 2020.
- [71] Irene Zhang, Naveen Kr. Sharma, Adriana Szekeres, Arvind Krishnamurthy, and Dan R. K. Ports. Building consistent transactions with inconsistent replication. In *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP '15*, page 263–278, New York, NY, USA, 2015. Association for Computing Machinery.
- [72] Ming Zhang, Yu Hua, Pengfei Zuo, and Lurong Liu. FORD: Fast one-sided RDMA-based distributed transactions for disaggregated persistent memory. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*, pages 51–68, Santa Clara, CA, February 2022. USENIX Association.
- [73] Jianjun Zheng, Qian Lin, Jiatao Xu, Cheng Wei, Chuwei Zeng, Pingan Yang, and Yunfan Zhang. Paxosstore: High-availability storage made practical in wechat. *Proc. VLDB Endow.*, 10(12):1730–1741, August 2017.

A Generalization of Pineapple

A.1 Handling Multi-key General Operations in One-shot Transactions

To handle multi-key operations, we need to modify Algorithm 2 in the following way:

- In the Get phase, send messages to read the value for each key in K_{input} .
- For each key in K_{input} , identify the value with largest pstamp from responses received from a quorum.
- Apply $f(\cdot)$ on the values identified in the previous step and generate output value for each key in K_{output} .
- All these output values will have the same pstamp, as the one generated in Line 6 in Algorithm 2.
- In the Put phase, contact a quorum to update the values.

The reason that this simple generalization works is because of the simplicity of Multi-Paxos. The existence of a leader simplifies the design. This is because the leader determines the order of the multi-key operation. Then later in the proof of linearizability, all the operations seen by the leader can be ordered beforehand.

On the other hand, in the case of Gryff, nodes make joint decision by using a dependency graph. It is not clear how they can jointly come to conclusion on the set of values that are the most recent ones for K_{input} and can be used by the one-shot transaction.

A.2 Handling Leader Change

To handle leader change or failures, we rely on the full version of Multi-Paxos to identify the leader. Moreover, we need to modify reads so that they do not return inconsistent values. Concretely, we need to make the following modifications:

- Include the ballot number and leader ID in the pstamp.
- Upon learning a larger ballot, the (potentially old) leader N_l knows that there exists another proposer that aims to be the leader. Hence, N_l needs to run the phase 1 of Paxos [36, 39] to elect a new leader.
- The comparison of pstamp needs to include the ballot number. In other words, any message from the old leader should be discarded (or reply with an NACK).
- Reads need to take extra caution, as if we adopt the same ABD reads, then read could return inconsistent results. This is because during leader change, a one-shot transaction from the old leader might get overwritten by the new leader.

To avoid this scenario, the reads need to back off and retry upon learning the existence of a new leader or a new proposer that is competing for becoming a leader. Reads can observe this from the ballot field in the received messages.

With this modification, a read is completed only if there exists a quorum that shares the same leader with the read and the same value-pstamp pair with the read.

Note that during the leader change, the one-shot transaction (from the old leader) might not be completed yet, as the Put phase has not get a confirmation from a quorum. However, it does not affect correctness, this is because as new leader completes phase 1 of Paxos, it will either learn and adopt the uncompleted one-shot transaction as its command for this slot, or it proposes a new one-shot transaction command. In the latter case, it means that there is no completed read observe the previously uncompleted one-shot transaction.

B Correctness Proof

Here, we prove that the algorithms presented in Algorithms 1 and 2 are correct. To prove linearizability, we need to show that under any execution of the algorithm, we can find a total ordering S which satisfies both the real-time constraint order and the semantic of the operations (i.e., the ordering is legal). Moreover, it satisfies liveness with partial synchrony.

B.1 Intuition

We use the pstamps to order operations. We start with the non-stable ordering of writes. After we place them in the order, we need to find the position for each RMW. Lastly, we find the place for each read.

To prove that the sequence we find following pstamps is correct, we need to show that pstamps correctly reflect real-time ordering and the semantics. This roughly follows the design of quorum intersection (of the Get phase), and the choice of using a single leader (in the Multi-Paxos component).

B.2 Proof

Given an execution of operations H , where operations may interleave with each other (as there are concurrent operations), we need to identify an ordering (or a sequence) S in which operations do not overlap with each other, i.e., the ordering that could occur on a single machine. Following the prior literature [8, 41, 42], we only need to consider completed operations. In the following proof, we assume that all operations are on a single key, as it is straightforward to extend the proof to multi-key operations.

Now we describe the following steps to find the position for each operation in S :

- Start with an empty sequence S .
- Order the writes using pstamps.
- For each RMW r that has a pstamp $(tag_r, slot_r)$, place them one by one, following the order of the slot index. There are only three possible cases:
 - tag_r is $(0, 0)$ and $slot_r = 1$, meaning that this RMW is the first non-read operation: RMW r is placed first in S (before all writes);
 - There is another RMW m such that $tag_m = tag_r$ and $slot_m = slot_r - 1$: RMW r is placed right after m in S ; and
 - Otherwise: in this case, there must exists a write w such that $tag_w = tag_r$: RMW r is placed right after w in S .

Note that because of the guarantee of Multi-Paxos, there cannot exist two RMWs that have the same slot index but different pstamp.

- For each read o , since we consider only the completed operation, it must return a null value (the default value), a value either from a write, or a value from a RMW.
 - For a null value, o is placed first in S (before any writes or RMWs);
 - For a value from write w , o is placed right after w in S ;
 - For a value from RMW r , o is placed right after r in S .

These steps place all the (completed) operations in S . Next we need to show that S is the total ordering that we are looking for.

Lemma 1. S satisfies real-time ordering.

Proof. This follows from the construction of pstamps. If an operation o_1 occurs before operation o_2 , then o_2 must have the same pstamp (this only happens when o_2 is a read) or a larger pstamp. The reason is that any completed operation is observed by a quorum (due to the Put phase); hence, when o_2 starts, the Get phase for all three operations allows o_2 to observe o_1 . Then the pstamp needs to be increased if o_2 is a write or a RMW. $\square$

Lemma 2. S satisfies the semantics of the interface defined in Section 2.1. That is, S is legal.

Proof. First, we need to show that any read d returns the most recent value by a write w or a RMW r in S . This follows from how we find the position for d . In other words, because of how we place d right after the value it returns, there is no non-read operations between w and d or between w and r .

Note that between w and d and between r and d , there could be other reads, but these reads do not matter, since they do not change values.

Second, we need to show that RMWs are consistent. That is, the value after applying RMW is correctly observed by future reads. The proof is by contradiction. Assume that there are four operations: two writes w_1, w_2 , one RMW r , and one read d such that

- w_1 is before w_2 in S ;
- r is right after w_2 in S ;⁵
- d is right after r in S ;
- rd sees the effect of r does a RMW on w_1 – whereas a legal execution should have r does a RMW on w_2 . This is because w_2 is the most recent write for RMW r , with respect to S .

The reason that we place r right after w_2 is that they have the same tag in associated pstamps. From the Get phase of Pineapple, this means that after this phase, the largest pstamp observed by the leader is from w_2 . As a result, the v^{new} is the computation after applying $f(\cdot)$ on the value of w_2 , not w_1 . This implies that the read d cannot observe the effect of r does a RMW on w_1 , a contradiction. This proves the lemma. $\square$

The two lemmas above prove that Pineapple satisfies linearizability.

Finally, we prove liveness.

Lemma 3. *Each operation completes in Pineapple when there is a stable leader.*

Proof. When $n = 2f + 1$ and there is a stable leader, all the phases are non-blocking, as each node and the leader is always able to obtain a response from a quorum. Therefore, all the operation completes under this case. $\square$

Since under partial synchrony, there is always a stable leader in Multi-Paxos. This together with the lemma above implies that Pineapple satisfies liveness.

C Throughput in WAN

Figure 10 shows the maximum throughput achieved by all the systems, except Gryff, with 80 to 130 close-loop clients that batch operations. The clients batch 5000 operations, send to a node, and wait for the completion of all the operations, before proceeding to the next batch. We choose this design to properly load the systems without the machines being throttled by the communication bottleneck. This is also how [46, 61] test the throughput of EPaxos in WAN. We did not evaluate Gryff,

because the implementation of Gryff from [10] does not support client batching. As reported in [10], Gryff’s throughput is similar to EPaxos.

We use the same workloads that we used in Section 4.4.2 but with different numbers of clients. The exact workloads are specified in the caption of Figure 10. We make the following observations:

- In read-heavy workloads, “Multi-Paxos with lease” has best performance. This is because with lease, the leader does not need to process as many messages.
- In a read-heavy workload with 25% conflicts, PQR performs slightly better than Pineapple. This is because with a higher number of concurrent conflicting operations, Pineapple’s reads can no longer complete on the fast path. In contrast, PQR’s reads complete after 1 RTT, reducing the amount of message processing.
- In balanced workloads, Pineapple has 3x to 4x throughput improvement over the next closest competitor (PQR and “Multi-Paxos with lease”). This is because the leader in Paxos systems is saturated by writes and RMWs, whereas writes are evenly distributed in Pineapple.
- In balanced workloads, the throughput of PQR and “Multi-Paxos with lease” are only slightly larger than Multi-Paxos, because the leaders are already saturated and cannot no longer process reads as efficiently as in the read-heavy workloads.
- EPaxos in general has slightly better performance than Multi-Paxos, but has lower throughput than other systems. This is expected because the dependency grows much faster in WAN, as also reported in [10, 61].

⁵Again, there might be some other reads between w_2 and r , but we do not care about reads here.

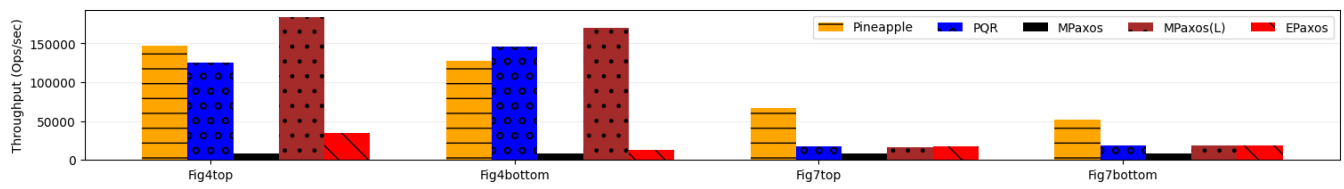


Figure 10: **Maximum attainable throughput in WAN.**

Fig4top: $n = 3$, 94.5% reads, 4.5% writes, 1% RMWs, 2% conflicts

Fig4bottom: $n = 3$, 94.5% reads, 4.5% writes, 1% RMWs, 25% conflicts

Fig7top: $n = 5$, 49.5% reads, 49.5% writes, 1% RMWs, 2% conflicts

Fig7bottom: $n = 5$, 49.5% reads, 49.5% writes, 1% RMWs, 25% conflicts